ContrastCheck - Revised Project Specification

Student: Heeral Chumber

Course: HCI 584

Revision purpose: Reduce Version 2 scope to a feasible, accurate accessibility checker.

Revision key

- **[UNCHANGED]** Retained from the original project direction.
- **[REVISED]** Changed to reflect implementation findings or remaining time.
- **[DEFERRED]** Removed from the required scope for this course version.
- **[PROPOSED - NICE TO HAVE]** Attempted only after required features are complete.

Project summary

[REVISED] ContrastCheck is a Python command-line tool that retrieves a Figma file through the Figma REST API and checks text/background color combinations against WCAG AA contrast guidance. It produces a saved report for designers before developer handoff. Version 2 prioritizes dependable contrast results and honest handling of uncertain layered backgrounds over a large graphical interface.

Problem and intended users

[UNCHANGED] Designers can miss low-contrast text because checking every screen and color pairing manually is repetitive. Problems may be found late during development or accessibility review.

[UNCHANGED] The primary user is a product or UI/UX designer checking a design before handoff. Developers and reviewers may also use the report.

Required inputs

- **[UNCHANGED]** A Figma file key or URL.
- **[REVISED]** A Figma personal access token supplied through the `FIGMA_TOKEN` environment variable. Tokens must never be committed to GitHub.
- **[PROPOSED - NICE TO HAVE]** A simple form for entering the file URL and token for a single run.

Required Version 2 features

1. Figma API connection

[UNCHANGED] Request the selected Figma file and handle successful and unsuccessful responses.

Acceptance criteria: The program reports a successful connection or a useful error without crashing. Missing credentials are explained clearly.

2. Recursive text-layer extraction

[UNCHANGED] Scan every page and nested child node to locate text layers and solid foreground colors.

Acceptance criteria: Text inside nested frames and components is included. Nodes without usable solid text fills are skipped or marked for review.

3. Background identification

[REVISED] Support the nearest solid-filled ancestor as the first reliable background rule. Do not claim exact detection for every layered design.

Acceptance criteria: Each result contains a detected solid background or an explicit "manual review required" reason. Gradients, images, opacity, and ambiguous overlapping layers may be marked for manual review.

4. WCAG contrast calculation

[UNCHANGED] Convert RGB values to relative luminance, calculate the contrast ratio, and classify normal text using the WCAG AA 4.5:1 threshold.

[REVISED] Large-text classification is outside the required scope unless font-size data can be interpreted and tested reliably.

Acceptance criteria: Known black/white and threshold test cases produce expected results.

5. Report generation

[REVISED] Generate a command-line and text-file report containing the text sample, node ID, foreground color, detected background color, contrast ratio, and status. Summaries will distinguish pass, fail, skipped, and manual-review results.

Acceptance criteria: The report is readable, avoids duplicate text/color pairs, and is saved successfully.

6. Tests and documentation

[REVISED - NEW REQUIRED WORK] Add automated tests for the contrast formula, recursive extraction, duplicate behavior, missing fills, background matching, and missing configuration. Add setup and run instructions.

Acceptance criteria: Tests can be run with one documented command, and a new user can configure the token without editing source code.

Deferred and optional features

Full Flask web application

[DEFERRED] The original specification proposed a Flask page as the main interface. A full web application is not required for the course version because correct analysis and testing have higher priority.

[PROPOSED - NICE TO HAVE] If the command-line version is complete, add a minimal local form that accepts a Figma URL and displays the generated summary.

Color-blindness simulation

[DEFERRED] Full-file visual simulation for multiple color-vision deficiencies is removed from the required scope. It introduces additional math, image rendering, and validation work that could weaken the core contrast checker.

[PROPOSED - NICE TO HAVE] Test one documented simulation method on selected foreground/background pairs after all required milestones pass.

Automatic replacement-color suggestions

[DEFERRED] Automatically proposing a replacement shade is not required. A suggestion must preserve design intent and should not be presented as correct without validation.

Processing flow

1. Read configuration and validate the token and file key.
2. Retrieve the Figma document JSON.
3. Walk pages and nested nodes recursively.
4. Extract text colors and parent/background context.
5. Calculate contrast when both colors are reliable.
6. Mark ambiguous cases for manual review.
7. Remove duplicate display entries.
8. Print and save the report and summary.

Error and uncertainty handling

[REVISED] The tool will distinguish an application error from an uncertain design case:

- Authentication, network, or missing-file errors stop the run with a useful message.
- Missing solid foreground colors are skipped with a reason.
- Ambiguous backgrounds are reported as manual review rather than pass or fail.
- Empty or unsupported files produce a valid zero-result summary rather than a crash.

Version 2 milestone order

1. Implement nearest-solid-background matching and manual-review states.
2. Add automated tests and small known Figma examples.
3. Improve report detail and summaries.
4. Improve user input, setup instructions, and documentation.
5. Attempt a minimal interface or color-blindness prototype only if required work is complete.

Definition of a successful course version

The project is successful when a designer can configure a token securely, analyze a Figma file, receive trustworthy WCAG contrast results for supported solid-color cases, see clear manual-review warnings for unsupported cases, and save a readable report. A web interface and color-blindness simulation are not necessary for success.